

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

TR-DLT-001: Load Customer and Order Data into Delta Tables

Related Functional Requirement(s): FR-DLT-001

Objective: Implement a Delta Live Table pipeline to load customer and order data from CSV files into Delta tables.

Target Cluster Configuration:

- Cluster Name: Gen AI POC Databricks Cluster
- Databricks Runtime Version: 10.4.x-scala2.12
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 Standard_DS3_v2
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: delta-lake, spark-csv

Source Data Details:

- Dataset Name: Customer Data
- Location: `/Volumes/gen\_ai\_poc\_databrickscOE/sdlc\_wizard/customerdata`
- Format: CSV
- Description: Customer data in CSV format
- Dataset Name: Order Data
- Location: `/Volumes/gen\_ai\_poc\_databrickscOE/sdlc\_wizard/orderdata`
- Format: CSV
- Description: Order data in CSV format

Target Data Details:

- Output Name: Customer Delta Table
- Location: `gen\_ai\_poc\_databrickscOE.sdlc\_wizard.customer`
- Format: Delta
- Description: Customer data in Delta format
- Output Name: Order Delta Table

- Location: ``gen_ai_poc_databrickscoe.sdlc_wizard.order``
- Format: Delta
- Description: Order data in Delta format

Job Flow / Pipeline Stages:

- Read customer CSV data into a Delta table named "customer"
- Read order CSV data into a Delta table named "order"

Data Transformations / Business Logic:

- Step: Read CSV data into Delta tables
- Description: Read CSV data from specified locations into Delta tables
- Transformation Logic: Use ``spark.read.csv()`` to read CSV data and ``df.write.format("delta").saveAsTable()`` to write to Delta tables

Error Handling and Logging:

- Log errors to a specified location using ``spark.sparkContext.setLogLevel("ERROR")``
- Handle errors using ``try-except`` blocks and log exceptions

TR-DLT-002: Transform Order Data and Remove Null/Duplicate Records

Related Functional Requirement(s): FR-DLT-002

Objective: Implement data transformations to add a new column "TotalAmount" to the "order" table and remove null and duplicate records from both "customer" and "order" tables.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Same as TR-DLT-001

Target Data Details: Same as TR-DLT-001

Job Flow / Pipeline Stages:

- Add a new column "TotalAmount" to the "order" table
- Remove null records from both "customer" and "order" tables
- Remove duplicate records from both "customer" and "order" tables

Data Transformations / Business Logic:

- Step: Add new column "TotalAmount"
- Description: Multiply "PricePerUnit" and "Qty" column values to calculate "TotalAmount"
- Transformation Logic: Use ``df.withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))``
- Step: Remove null records
- Description: Remove rows with null values from both "customer" and "order" tables
- Transformation Logic: Use ``df.filter(df["column_name"].isNotNull())``

- Step: Remove duplicate records
- Description: Remove duplicate rows from both "customer" and "order" tables
- Transformation Logic: Use `df.dropDuplicates()`

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-003: Create ordersummary Table and Load Joined Data

Related Functional Requirement(s): FR-DLT-003

Objective: Implement a data pipeline to create an "ordersummary" table and load joined data from "customer" and "order" tables into it using SCD type 2.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: Customer Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdlc_wizard.customer``
- Format: Delta
- Description: Customer data in Delta format
- Dataset Name: Order Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdlc_wizard.order``
- Format: Delta
- Description: Order data in Delta format

Target Data Details:

- Output Name: ordersummary Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdlc_wizard.ordersummary``
- Format: Delta
- Description: Joined customer and order data in Delta format

Job Flow / Pipeline Stages:

- Create the "ordersummary" table if it does not exist
- Join "customer" and "order" data using the "CustId" field
- Load the joined data into the "ordersummary" table using SCD type 2

Data Transformations / Business Logic:

- Step: Join customer and order data
- Description: Join "customer" and "order" tables on "CustId"
- Transformation Logic: Use `df_customer.join(df_order, "CustId")``
- Step: Load joined data into ordersummary table
- Description: Load joined data into "ordersummary" table using SCD type 2

- Transformation Logic: Use ``df_joined.write.format("delta").option("mergeSchema", "true").saveAsTable()``

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-004: Update ordersummary Table on Customer Data Change

Related Functional Requirement(s): FR-DLT-004

Objective: Implement a data pipeline to update the "ordersummary" table whenever there is a change in the "customer" table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: Customer Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdmc_wizard.customer``
- Format: Delta
- Description: Customer data in Delta format

Target Data Details:

- Output Name: ordersummary Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdmc_wizard.ordersummary``
- Format: Delta
- Description: Joined customer and order data in Delta format

Job Flow / Pipeline Stages:

- Detect changes in the "customer" table
- Update the "ordersummary" table by making old records inactive and new records active
- Update the "StartDate" and "EndDate" columns accordingly to maintain history

Data Transformations / Business Logic:

- Step: Detect changes in customer data
- Description: Detect changes in "customer" table
- Transformation Logic: Use ``df_customer.join(df_ordersummary, "CustId", "leftanti")``
- Step: Update ordersummary table
- Description: Update "ordersummary" table by making old records inactive and new records active
- Transformation Logic: Use ``df_ordersummary.write.format("delta").option("mergeSchema", "true").saveAsTable()``

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-005: Create customeraggregatespend Table and Load Aggregated Data

Related Functional Requirement(s): FR-DLT-005

Objective: Implement a data pipeline to create a "customeraggregatespend" table and load aggregated data from the "ordersummary" table into it.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: ordersummary Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdmc_wizard.ordersummary``
- Format: Delta
- Description: Joined customer and order data in Delta format

Target Data Details:

- Output Name: customeraggregatespend Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdmc_wizard.customeraggregatespend``
- Format: Delta
- Description: Aggregated customer spend data in Delta format

Job Flow / Pipeline Stages:

- Create the "customeraggregatespend" table if it does not exist
- Aggregate the "TotalAmount" column from the "ordersummary" table by grouping by "Name" and "Date" columns
- Load the aggregated data into the "customeraggregatespend" table

Data Transformations / Business Logic:

- Step: Aggregate data
- Description: Aggregate "TotalAmount" column by grouping by "Name" and "Date" columns
- Transformation Logic: Use ``df_ordersummary.groupBy("Name", "Date").agg(sum("TotalAmount"))``
- Step: Load aggregated data into customeraggregatespend table
- Description: Load aggregated data into "customeraggregatespend" table
- Transformation Logic: Use ``df_aggregated.write.format("delta").saveAsTable()``

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-006: Implement using Delta Live Tables

Related Functional Requirement(s): FR-DLT-006

Objective: Implement the data pipeline using Delta Live Tables to perform the required data transformations and loading.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details: Same as previous TRs

Target Data Details: Same as previous TRs

Job Flow / Pipeline Stages:

- Implement the data pipeline using Delta Live Tables

Data Transformations / Business Logic:

- Step: Implement Delta Live Tables pipeline
- Description: Implement the data pipeline using Delta Live Tables
- Transformation Logic: Use ``@dlt.table`` and ``dlt.read()`` to implement the pipeline

Error Handling and Logging: Same as TR-DLT-001